

A Formal Approach to Workflow Analysis

Author(s): Amit Basu and Robert W. Blanning

Source: *Information Systems Research*, Vol. 11, No. 1 (March 2000), pp. 17-36

Published by: INFORMS

Stable URL: <http://www.jstor.org/stable/23015971>

Accessed: 10-11-2016 12:50 UTC

JSTOR is a not-for-profit service that helps scholars, researchers, and students discover, use, and build upon a wide range of content in a trusted digital archive. We use information technology and tools to increase productivity and facilitate new forms of scholarship. For more information about JSTOR, please contact support@jstor.org.

Your use of the JSTOR archive indicates your acceptance of the Terms & Conditions of Use, available at
<http://about.jstor.org/terms>



INFORMS is collaborating with JSTOR to digitize, preserve and extend access to *Information Systems Research*

A Formal Approach to Workflow Analysis

Amit Basu • Robert W. Blanning

Owen Graduate School of Management, Vanderbilt University, Nashville, Tennessee 37203

Amit.Basu@owen.vanderbilt.edu

Bob.Blanning@owen.vanderbilt.edu

Agile manufacturing, fast-response micromarketing, and the rise of the virtual organization have led managers to focus on cross-functional business processes that link various divisions and organizations. These processes may be realized as one or more workflows, each of which is an instantiation of a process under certain conditions. Because an ability to adapt processes to workflow conditions is essential for organizational responsiveness, identifying and analyzing significant workflows is an important activity for managers, organization designers, and information systems specialists. A variety of software systems have been developed to aid in the structuring and implementation of workflow systems, but they are mostly visualization tools with few analytical capabilities. For example, they do not allow their users to easily determine which information elements are needed to compute other information elements, whether certain tasks depend on other tasks, and how resource availability affects information and tasks. Analyses of this type can be performed by inspection, but this gives rise to the possibility of error, especially in large systems. In this paper, we show how a mathematical construct called a metagraph can be used to represent workflows, so that such questions can be addressed through formal operations, leading to more effective design of organizational processes.

(Workflows; Metagraphs; Information Interactions; Task Interactions; Resource Interactions)

1. Introduction

Most organizations have traditionally been organized by function, such as purchasing, manufacturing, marketing, engineering, and accounting. This practice continues: Human, physical, and financial resources are often managed by function, and most coordination is intrafunctional rather than interfunctional. However, many organizations are finding that they must also manage processes—such as order fulfillment, new product introduction, and interorganizational supply chain management—that span their separate functional units and that integrate their activities with those of other organizations. These processes are essential to the well-being of organizations in a dynamic competitive environment (Davenport 1993, Marschak 1995, Thomson 1995), which is beginning to result in

the development of models of business processes (Barua et al. 1996).

Processes must be adapted to specific circumstances, such as inventory outages or negative responses to new products. The specific collection of tasks, resources and information elements involved in each such circumstance comprise a workflow.¹ Several persons will be involved in the management of processes and their workflows (Hammer and Champy 1993, Khoshafian and Buckiewicz 1995, Hammer 1996). Senior executives need to understand what information elements are required and produced by a process and what resources are needed. Process managers need to understand how the tasks in each workflow interact with each other through the information elements they

¹A more restricted definition limits workflow to only automated activities in a process (WFMC 1994).

use and produce. Information technology managers need to understand how information elements, tasks, and resources interact, so that effective operational and decision support systems can be designed. This suggests a need for modeling processes in a way that facilitates identification and analysis of their component workflows.

Processes and their associated workflows can be modeled in several ways using different tools. Four perspectives commonly used in process representation are as follows (Curtis et al. 1992, Kwan and Balasubramanian 1997):

1. *Informational modeling* focuses on the informational entities involved in the process, the structure of these entities, and their interrelationships.

2. *Functional modeling* focuses on what tasks are being performed and what informational elements are involved in these tasks.

3. *Organizational modeling* focuses on what agents/resources are involved in each task, where information entities are stored, and what communication is needed between agents/resources.

4. *Transactional modeling* examines issues of timing (sequencing) and control, both within and between the tasks involved in the process.²

Traditionally, these perspectives have been implemented separately. A significant contribution of our approach is that it integrates the informational, functional, and organizational perspectives within a single model. This allows not only graphical visualization of processes, but also their formal analysis because the model is based on a powerful graph-theoretic construct, a *metagraph*. This construct extends the features offered by traditional graph structures such as digraphs and hypergraphs, and allows us to address questions such as the following using algebraic operations on metagraph representations of processes:

1. How do information elements relate to each other through the tasks that use and produce them—for example, which information elements are needed to determine other information elements, and which information elements are intermediate elements in a

process that calculates other elements? (*informational modeling*)

2. How do tasks relate to each other through the information elements that they use and produce—for example, if a task is disabled, what other tasks cannot be executed? (*functional modeling*)

3. How do resources relate to each other through the tasks that use them and the information used and produced by these tasks—for example, what information passes from one resource to another and if a resource is unavailable, what other resources are affected? (*organizational modeling*)

In addition to such questions within each perspective, there are other questions that span them. For instance, if a particular resource were to become unavailable, how would the workflows in the process be affected? A significant aspect of the integration enabled in our approach is that such questions can also be addressed in a structured manner.

While we do not explicitly address transactional issues of timing and scheduling in this paper, these issues could be addressed by including task duration and temporal constraints as additional metagraph edge attributes.

The contribution of this paper is twofold. First, it extends the theory of metagraphs, as developed in prior work (Basu and Blanning 1994a, 1994b, 1997, 1998) by allowing representation of resources and introducing two transformations that are useful for workflow analysis, namely the inverse metagraph and the element flow metagraph. Second, it uses these operators in the context of workflow metagraphs to create three alternative but equivalent representations, an information-centric view, a task-centric (or functional) view in terms of the task interaction metagraph, and a resource-centric (or organizational) view in terms of the resource interaction metagraph. Exploiting the fact that each of these views is a metagraph, we then show how various analytical properties and operators defined for metagraphs can be applied to analyze the roles and interactions of different workflow components to answer important questions in workflow and process design, such as those listed above.

We begin in § 2 by examining workflow modeling approaches and elements of workflows. In § 3, we review some essential features of metagraphs and define

²Curtis et al. (1992) use the term “behavioral modeling” for this perspective, but we feel that “transactional modeling” is more representative.

the new constructs. In § 4 we show how metagraphs can be used to model and analyze workflows, using an example to illustrate our approach.

2. Workflow Modeling

2.1. Existing Approaches

There is a growing number of commercially available computer-based workflow management tools (Data-mation 1995, Silver 1995, Georgakopoulos et al. 1995) and proprietary tools crafted into specialized workflow systems (Morschheuser et al. 1996). These tools facilitate graphical and iconic representation of workflows for purposes of visualization, and in some cases they regulate the flow of documents between workstations. Unfortunately, there is no common understanding of what functionality such a tool should have or how the different tools should relate to each other. Therefore, toolmakers have to rely on *ad hoc* judgments regarding the functionality they should build into their tools, and tool-users have to rely on *ad hoc* judgments in selecting and integrating tools into their workflow processes.

In addition, although these tools facilitate workflow specification and visualization, and some even allow simulation of processes under various conditions, they usually do not support formal workflow analyses (Khoshafian and Buckiewicz 1995, Kouloupoulos 1995, Poyssick and Hannaford 1996). Questions concerning the relationships among information elements, tasks, and resources can be answered only through manual inspection and reasoning. This is acceptable for small organizations and simple workflows, but even then there is a possibility of error. Similarly, while process simulation can provide useful insight into process behavior, it does not address questions about the inter-relationships among process components.

What is needed is a theoretical framework for the representation, analysis, and manipulation of workflow systems. Although some organizations, such as the Workflow Management Coalition (WFMC 1994, Swenson 1995) and IBM (Mohan et al. 1995), have begun to create such frameworks, they are often narrative and diagrammatic in character and are not built on a solid analytical foundation. There are a few approaches that do have formal bases, such as Petri nets

(Murata 1989), colored Petri nets (Aalst 1995) and state charts (a good review of these and other process modeling approaches is provided in Curtis et al. 1992). The Petri net approach enables analytical solution of some important questions in workflow analysis, such as finite termination and identification of dangling tasks. However, Petri nets are primarily oriented to analysis of timing and conflict resolution considerations, rather than of workflow component connectivity and interactions. The STATEMATE tool (Harel et al. 1988), based on the state chart and higraph constructs developed in Harel (1987), has been applied to process modeling. However, as shown in Basu and Blanning (1994b), the complexity of state charts makes it difficult to formalize some important notions of connectivity, such as transitive connectivity between sets of elements. As we demonstrate in this paper, this is essential for effective workflow analysis.

Thus, there is a need for a more general framework that focuses on the connectivity of workflow systems and an analysis of the requirements of different tasks and workflows. The recent development of the Unified Modeling Language (UML) (Booch et al. 1998) reflects this need and represents an effort to address it. However, the UML is a complex collection of multiple representational and analytical tools. For instance, it includes forms of Petri nets and state charts as well as several other constructs. While it is clearly a comprehensive approach, it requires analysts and designers to learn multiple constructs and understand their interactions. We present here an alternative analytical framework based on a mathematical tool called metagraphs, which allows different components of processes to be represented both graphically and analytically, and enables effective connectivity and component interaction analysis of workflows, using a single representational construct.

2.2. Elements of Workflow

Consider the example of a loan evaluation process. The input to the process is a document containing certain information items describing the applicant and the characteristics of the loan being requested. Various tasks are used to analyze the application, possibly to request that additional information be made available,

Table 1 Relevant Questions About Process Components During Process Analysis

Process Component	Questions
Information Elements	1. Given two information elements, is one of them needed to determine the value of the other? Is it needed only under certain conditions and if so, what are the conditions? 2. Given two sets of information elements, is it possible to determine the value of the second set from the elements in the first set? If not, are there any additional information elements that would make it possible to do so? 3. Given a complex process, are there any ways to focus on only important information elements, hiding intermediate elements that are needed only to calculate the important ones?
Tasks	4. Given a task that we wish to execute, what other tasks must be executed to provide the information needed to do so? 5. If a task is disabled, what other tasks will be affected—that is, what other tasks cannot be executed?
Resources	6. Given a set of resources, what information passes among them as the tasks that utilize them are executed? 7. If a resource is unavailable, what other resources are affected—that is, what other resources will be idle because their tasks cannot be executed?
Interactions among Components	8. If a resource used in a process is unavailable, which workflows within the process can still be completed? 9. If an information element is found to be inaccurate, which resources were used, directly or indirectly, in the calculation of that element?

and to arrive at a decision. Human and computer resources, such as loan officers, loan managers, fax machines, and workstations are used to accomplish these tasks. The flow of information items, the scheduling of tasks, and the allocation of resources are often specified in procedures manuals and/or in automated workflow systems.

We now define the central concepts of workflow systems and present some of the questions pertinent to workflow analysis. We begin with the following definitions:

1. An *information element* is an atomic data item (e.g., a number, a character string, an image, or an icon) or a collection of atomic data items (as in a document).
2. A *report* is a collection of information elements.
3. A *task* is an ordered pair of reports, the first of which is an input to the task and the second of which is its output. A task is *executed* when the inputs are used to determine the output.
4. A *workflow system* is a set of information elements and a set of tasks, such that the inputs and outputs of the tasks are all in the set of information elements.
5. An *assumption* is a proposition (which may be true or false) associated with a task, such that the assumption must be true for the task to be executed. For example, it may be assumed that the dollar value of a transaction is less than a certain amount.

6. A *resource* is an entity associated with one or more tasks, and the resource must be available if the tasks are to be executed. Several resources may be associated with a single task, and vice versa. Resources may be people, workstations, categories of people (i.e., roles, such as programmer or file clerk), etc.

7. A *process* is a set of tasks that connects one set of information elements, called the *source*, to another set of information elements, called the *target*. All the inputs for any task in the process must be either in the source or in the output of some other task(s) in the process.

8. A *workflow* is a particular instantiation of a process. Because a process may include decision points that can cause the process to branch in different ways during execution, a process can contain several possible workflows, each corresponding to a particular set of values for all relevant branching conditions.

The purpose of constructing a framework for workflow management is that it allows us to formulate questions about the relationships among the three important components of workflows: information, tasks, and resources. Nine such questions are shown in Table 1 and are answered in the following sections of this paper.

To answer these questions we need an analytical

framework that allows us to: (1) capture all the important elements in the workflow process, and (2) address these questions by means of rigorous analytical procedures rather than visual inspection and intuition. The theory of metagraphs provides a basis for such a framework. In the next section, we summarize the major features of metagraphs and present some extensions to the theory that are pertinent to workflow analysis. In later sections, we will use these features to address questions like those listed above for workflow management.

3. Metagraphs

Metagraphs are graphical structures that represent directed relationships between sets of elements. They extend both directed graphs (by allowing multiple elements in vertices) and hypergraphs (by including directionality in edges) (Berge 1985, 1989). They model the features of modern information systems more naturally than these other constructs (Basu and Blanning 1994a). In this section, we describe metagraphs and present two new operations on metagraphs that are later used in workflow analysis.

3.1. Basic Features of Metagraphs

We begin by providing an informal overview of important features and properties of metagraphs in terms of examples; related definitions are included in Appendix A. Detailed discussion of this material is available in a number of references (Basu and Blanning 1994a and 1994b, 1997, 1998). Consider the metagraph illustrated in Figure 1, consisting of eight elements $X = \{x_1, \dots, x_8\}$, and four edges $E = \{e_1, e_2, e_3, e_4\}$. The set X of all elements in the metagraph is called the *generating set* of the metagraph. The edges describe directed

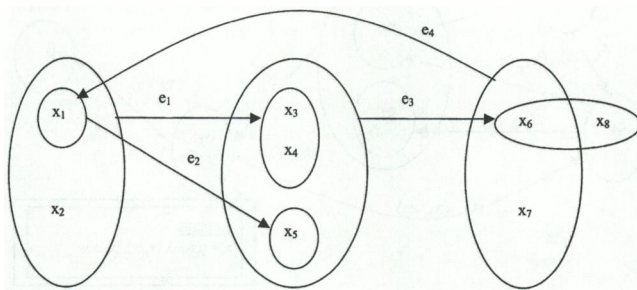
relationships between the sets of elements defined by each vertex (shown as an oval). Each edge can also be represented as an ordered pair of sets of elements; the first set is called the *invertex* V_e of the edge (e.g., the invertex of e_1 is the set $\{x_1, x_2\}$), and the second set is its *outvertex* W_e (e.g., the set $\{x_3, x_4\}$ for e_1). The *coinput* (*cooutput*) of each element in the invertex (outvertex) of an edge consists of the other elements in the invertex (outvertex). Thus, the coinput of x_1 in e_1 is $\{x_2\}$ while the cooutput of x_3 in that edge is $\{x_4\}$.

We next consider the concept of a path. There are two types of paths in metagraphs. Given two elements, a *simple path* from the first to the second is a sequence of edges that connect them accordingly. For example, the sequence of edges $\langle e_1, e_3 \rangle$ is a simple path from x_1 to x_6 . However, since a simple path is defined in terms of a single input and single output element, it can have a coinput and a cooutput, just as for a single edge. In our example, the coinput of x_1 in the path is $\{x_2, x_5\}$, while the cooutput of x_6 is $\{x_3, x_4, x_8\}$.

The concept of a simple path is insufficient to describe connectivity in a metagraph. For instance, in Figure 1 every simple path from x_1 to x_6 has a coinput that includes elements from the set $\{x_3, x_4, x_5\}$. Yet, it is apparent that x_1 and x_2 are sufficient to determine x_6 , if the edges e_1, e_2, e_3 are used; however, these edges do not comprise a simple path, since they do not form a sequence. To address this, the concept of a *metapath* is used. A metapath connects one set of elements (the source) to another set of elements (the target). Intuitively, a metapath from a source set B to a target set C is a set of edges such that, starting with only the elements in B , all the elements in C can be determined using these edges in some feasible (possibly parallel) order of instantiation. Thus, in Figure 1, the set $\{e_1, e_2, e_3\}$ comprises a metapath from $\{x_1, x_2\}$ to $\{x_6\}$.

Even though a metapath is defined as a set of edges, the sequential dependencies between its edges can be extracted and utilized, based on an algebraic representation of the metagraph. The structure of a metagraph can be represented by an adjacency matrix and operations such as addition, multiplication and transitive closure (Basu and Blanning 1994b). Analysis of the metagraph, including identification of paths and metapaths, can then be stated in terms of these operations and related procedures. Examples of analyses include

Figure 1 A Metagraph



identification of cycles, identification of dominant metapaths (those that contain no unnecessary edges or source elements), and identification of bridges (edges that are on all metapaths between a given source and target).

The entities (models, files, etc.) represented by metagraph edges may be constrained by assumptions. An assumption is a proposition—that is, a statement that may be either true or false—that appears in the invertex of an edge and that must be true for the edge to be used in a metapath. Assumptions appear in the generating set along with the other elements. Metagraphs in which propositions are included are called *conditional metagraphs* (note that a conditional metagraph is a generalization of a simple metagraph).

The values of different propositions can be used to specify alternative contexts for a conditional metagraph. Thus, if a set of propositions P is known to be true and another set Q is known to be false, then this knowledge can be used to simplify a conditional metagraph so that only those edges that are valid in that context are retained. This is a useful abstraction on metagraphs because it avoids the need to consider edges that cannot be used under the stated conditions. Another abstraction is provided by a projection operator, which simplifies a metagraph to represent relationships between a subset of the elements in the original generating set. In the projection, each edge may correspond to one or more edges from the base metagraph, or more generally, to one or more metapaths in the base metagraph.

The value of these abstractions is not only in their visualization benefits but also in that each of them results in a metagraph, which can be analyzed in the same way as the original metagraph. This is a crucial feature because all the operations and transformations that we use on metagraphs result in other metagraphs, which enables the same core set of analytical procedures to be applied to a variety of related representations of a given system.

3.2. Two New Operations

Two new operations on metagraphs that are useful for workflow representation and analysis are the *inverse metagraph* and the *element flow metagraph*.

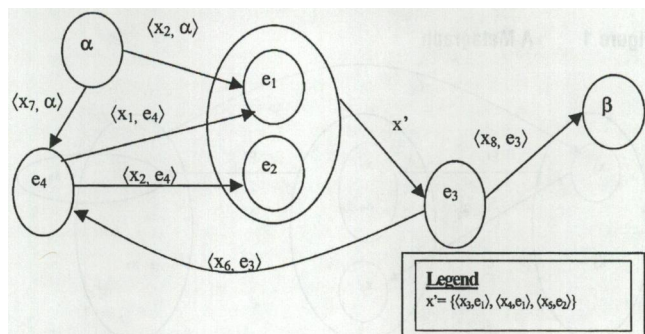
The inverse metagraph is a representation in which

the generating set is made up of edges from the original metagraph, and where the edges correspond to combinations of elements from the original metagraph's generating set. Thus, the inverse metagraph is intuitively like the dual of a simple or directed graph. As with these simple graph constructs, one could construct a dual metagraph by simply transposing the incidence matrix of the original metagraph. However, as shown in (Basu and Blanning (1997), the semantics of the resulting dual structure is different (in that the invertex of each edge represents a disjunction, rather than a conjunction, as in the primal metagraph). That is why we define the inverse metagraph, which provides the necessary "edge-centered" representation while retaining the conjunctive form of the edges.

DEFINITION. Given a metagraph $S = \langle X, E \rangle$ (the notation used here is explained in the appendix), its *inverse* $T = \langle X', E' \rangle$ is a metagraph such that $X' = E \cup \{\alpha, \beta\}$, where α denotes the external source, β denotes the external target, and $e' \in E'$ iff in the primal metagraph, the primal elements corresponding to e' are in the outvertex of the primal edges in $V_{e'}$ and also in the invertex of the primal edges in $W_{e'}$. In addition (1) all pure inputs (i.e., elements that are not in any outvertex) in the primal appear in the inverse metagraph as edges from α , and (2) all pure primal outputs (i.e., elements that are not in any invertex) appear in the inverse metagraph as edges to β .

For example, the inverse of the metagraph in Figure 1 is the metagraph in Figure 2. Because both representations are metagraphs, properties such as paths, metapaths, cycles, and bridges can be applied to the inverse metagraph just as to the primal, and the same

Figure 2 The Inverse Metagraph for Figure 1



algebraic operations and procedures can be applied to both. Thus, the inverse metagraph provides a complementary visual representation of a system that still supports metagraph analysis.

The inverse of a given metagraph can be generated from its incidence matrix G . This is a matrix whose rows correspond to the elements in the metagraph's generating set and whose columns correspond to the edges in the metagraph. There is a "+1" (resp., "-1") entry whenever the row element is an output (resp., input) of the edge corresponding to the column; all other entries are zero.

The inverse metagraph can be constructed using the following procedure:

Procedure Inverse

1. For each column j of G , form all combinations of columns k such that $g_{ij} = -1$ and $g_{ik} = +1$, selecting no more than one column from each such row, and create an edge with each of the "+1" column indices in its invertex and each of the "-1" column indices in its outvertex. Label the edge with the set of row-column index pairs used to construct the edge (e.g., if entries g_{ij} and g_{mn} are used for the invertices, and column p is used for the outvertex of edge y , then $label(y) = \{\langle x_i, e_j \rangle, \langle x_m, e_n \rangle\}$).

2. If a set of two or more edges have the same invertices and labels, then replace all these edges with a single edge having the same invertex and label, and the union of all the component outvertices in the outvertex.

3. For each row j in G that has only -1 (resp., +1) entries, create a single edge to (resp., from) the column entries from (resp., to) α (resp., β), with the label $\{\langle x_i, \alpha \rangle, \langle x_j, e_k \rangle\}$, for each column e_k with a "+1".

For the metagraph in Figure 1, the matrix G is as follows, and the algorithm would proceed as follows:

1. The four edges $\langle \{e_4\}, \{e_1\} \rangle$ (with label $\langle x_1, e_4 \rangle$), $\langle \{e_4\}, \{e_2\} \rangle$ (with label $\langle x_2, e_4 \rangle$), $\langle \{e_1, e_2\}, \{e_3\} \rangle$ (with label $\langle x_3, e_1 \rangle$), $\langle x_4, e_1 \rangle, \langle x_5, e_2 \rangle$ and $\langle \{e_3\}, \{e_4\} \rangle$ (with label $\langle x_6, e_3 \rangle$), are identified.

2. The edges $\langle \alpha, \{e_1\} \rangle$ (with label $\langle x_2, \alpha \rangle$), $\langle \alpha, \{e_4\} \rangle$ (with label $\langle x_7, \alpha \rangle$), and $\langle \{e_1\}, \beta \rangle$ (with label $\langle x_8, e_3 \rangle$) are added.

The result is the inverse metagraph in Figure 2. Note that the inverse is equivalent to the original metagraph, in that the latter can be reconstructed given the former.

G	e_1	e_2	e_3	e_4
x_1	-1	-1	0	+1
x_2	-1	0	0	0
x_3	+1	0	-1	0
x_4	+1	0	-1	0
x_5	0	+1	-1	0
x_6	0	0	+1	-1
x_7	0	0	0	-1
x_8	0	0	+1	0

Another operation, one that focuses attention on the flow of particular primal elements, is the transformation of a primal metagraph into its corresponding *element flow metagraph* (EFM). We first define the EFM, provide some intuition on its structure, specify an algorithm for generating it for a given metagraph, and then illustrate the algorithm with an example.

DEFINITION. Given a metagraph $S = \langle X, E \rangle$ and a specific subset X' of X , the element flow metagraph corresponding to X' is a metagraph $S = \langle X', F \rangle$ in which for each edge $f = \langle V_f, W_f \rangle \in F$, there exist edges $e_1, e_2 \in E$ such that

1. $V_f \subseteq V_{e_1}$,
2. $W_{e_1} \cap V_{e_2} = Z \neq \emptyset$, and
3. $W_f \subseteq V_{e_2}$.

The set of elements Z is the *flow content* on f through the (primal) edge pair e_1, e_2 from V_f to W_f . Because there could be several edge pairs $e_1, e_2 \in E$ corresponding to the same $f \in F$, we also define the *flow composition* $C(f)$ as the set of all such edge pairs representing flow on f .

Informally, this transformation is a simplified form of the primal in which the generating set consists of X' , and each edge e' identifies what elements in $X \setminus X'$ flow from vertices in S containing the elements in $V_{e'}$ to elements in vertices containing the elements in $W_{e'}$. Thus, the edges represent a dependency between the elements in X' . However, this dependency is not of the type represented by the projection operator, which identifies metapaths between vertices. An edge e' in the EFM represents the fact that there is some edge in S whose invertex contains the elements in $W_{e'}$, and

which requires input from some edge whose invertex contains the elements in V_e .

For example, the EFM defined for the elements x_2 , x_4 and x_7 from the metagraph in Figure 1 is shown in Figure 3. The composition of each edge is shown in the legend, where the content of each edge pair is included in parentheses. Once again, as with the inverse, the EFM can be analyzed using metagraph analytical procedures. For examples, cycles can be identified and analyzed, metapaths can be identified to identify the scope of the impact of one set of resources upon another, and bridges can be used to identify critical element flows. However, because the edges in the EFM represent only direct element flows (across a single edge in the original metagraph), transitive element flows are captured only when the set X' covers all the edges in the original metagraph (in the sense that for $e \in E$, $X' \cap V_e \neq \emptyset$).

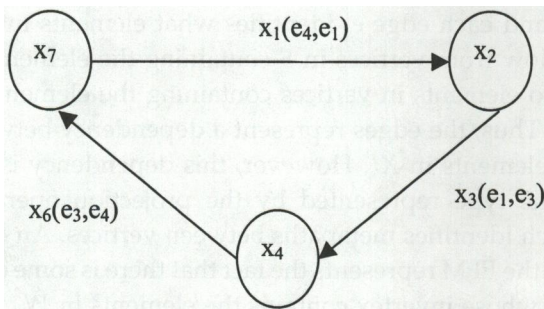
While the EFM could be used for any subset of X , it is particularly useful when X' is a separate type of element. For instance, in § 4 we will show how this transformation is useful when X' represents a set of resources. It directly provides a view that represents the interaction between resources.

The EFM can be generated from a given metagraph by the following Procedure EFM, which uses the element task incidence matrix G . Let G_1 be the submatrix of G corresponding to the elements in X' and G_2 be the sub-matrix of G corresponding to elements in $X \setminus X'$ that are not either pure inputs or pure outputs. Then the procedure is defined as follows:

Procedure EFM

1. Perform the operation $G_2 \otimes G_1^T$ (the $\otimes$ operator is

Figure 3 The Element Flow Metagraph for Figure 1 with $X' = \{x_2, x_4, x_7\}$



defined in Appendix A, and G_1^T is the transpose of G_1); the result is a matrix R whose rows correspond to the nonterminal elements in $X \setminus X'$ and whose columns correspond to the elements in X' .

2. For each row r_i of R corresponding to element x_i , construct edges as follows:

- a) For each edge t_a (resp., t_b) appearing as a positive (resp., negative) entry in any cells in r_i , create an invertex (resp., outvertex) consisting of all column indexes corresponding to the columns z_j such that $t_a \in r_{ij}$ (resp., $-t_b \in r_{ij}$).

- b) Combine each such invertex and outvertex pair as an edge. The flow composition of the edge is defined as $\{r_i \langle t_a, t_b \rangle\}$.

- c) Combine all edges with the same vertices into a single edge whose flow composition is the union of the flow composition of the component edges.

The procedure can be applied to the metagraph in Figure 1 to generate the EFM in Figure 3 for $X' = \{x_2, x_4, x_7\}$ as follows:

Step 1. Based on the partitioning of the incidence matrix with x_2, x_4, x_7 for G_1 and x_1, x_3, x_5, x_6 for G_2 , the R matrix is as follows:

R	x_2	x_4	x_7
x_1	$-e_1$	0	$+e_4$
x_3	$+e_1$	$-e_3$	0
x_5	0	$-e_3$	0
x_6	0	$+e_3$	$-e_4$

Step 2: From the row for x_1 , we get the edge $e' = \langle x_7, x_2 \rangle$ with flow composition $\{x_1 \langle e_4, e_1 \rangle\}$; similarly, we get the edges e'' and e''' from rows x_3 and x_6 respectively.

Note that there is no edge corresponding to the element x_5 . This is because the edge that produces that element is not covered by X' . This illustrates why the covering condition stated earlier is important in order for an EFM to capture all element flows, both direct and indirect, among the elements of X' .

4. Metagraph-Based Workflow Analysis

In this section, we discuss how the representation of processes and their component workflows in terms of

metagraphs can help business analysts address important workflow analysis questions, such as those mentioned in § 2.2. We start by showing how the various elements of a workflow system can be represented using metagraphs. An extended example of a workflow metagraph is described next. Then, in the remainder of the section, we examine how metagraph analysis can be applied to three distinct views of workflows, each focusing on specific components of workflows, namely information elements, tasks and resources respectively.

4.1. Metagraph Representation of Workflows

Each information element in a workflow can be represented as an element of the generating set X , or more specifically, of X_v . A collection of information elements comprising a report can then be represented as a vertex. This presumes that each report is either the input or output of some task, which is reasonable for black box analysis of interactions among tasks, as discussed below. Each task is itself represented as an edge in the metagraph. We assume that the input to each task is a report, as is its output; and as before, this assumption is reasonable, because the report comprising each task's input can be composed of elements from one or more reports (and/or manual inputs from some resource).

It follows, then, that each process can be represented by a metagraph. More generally, a metagraph can be used to represent the tasks comprising a collection of possibly related (or overlapping) workflows comprising the process. This last point is significant, because most other graph constructs do not allow such overlapped representation. The collective representation of multiple workflows in a single metagraph enables analysis and possible redesign of these workflows in a more comprehensive manner.

Metagraphs, like other modeling techniques based on graph theory, provide a "black box" representation of reality. That is, a task, as represented by a metagraph edge, is viewed as a pair of inputs and outputs. There is no consideration of what happens inside the black box. Thus, a metagraph representation of a task does not take into consideration its duration, the quality of work done, the value added by the task to any process, and the type of monitoring and control needed to detect errors in the task.

Although metagraphs are limited in this regard, they do have a powerful advantage. First, metagraphs model the essential structure of a workflow system, in that they allow for an explicit representation of the components of the system and the interactions among them. Thus, they can be used to determine what sorts of information can be furnished by a workflow system, given the information processing capabilities of its components. Second, as we will see in this paper, metagraphs allow for multiple views of a workflow system—an element-centric view, a task-centric view, and a resource-centric view. Because each of these views is itself a metagraph, the same mathematical machinery can be used for all of them. Third, it is possible that metagraphs can be extended to include some of the contents of the black box mentioned above, for example, by labeling the edges with estimates of costs, durations, error rates, etc. However, such an extension is beyond the scope of this paper.

To complete the representation, a workflow can be represented as a metapath from a set of information elements comprising a source to another set comprising the target. Assumptions underlying each task can also be represented in the metagraph, by augmenting the generating set with a set X_p of propositions and including the relevant propositions in the invertices of the task edges. And finally, resources needed for each task can also be represented, by further augmenting the generating set with a set X_r of resources. Then, the resources required in each task can be represented as additional inputs to the corresponding edge. Note that the separation of the generating set into the three component sets X_v , X_p , X_r is not merely for convenience. The primary motivation for this is that the evaluation of elements from each set is different. While information elements can have any value from their particular domain, propositions evaluate to either "true" or "false" (with a task becoming viable only if all its assumptions evaluate to "true") and resources evaluating to either "available" or "unavailable" (with a task becoming viable only if all its resources are available). From a visualization perspective, the assumptions underlying a task and the resources it needs can be presented to the user as labels on the edge itself, rather than as invertex assumptions. However, the invertex representation may be more intuitive.

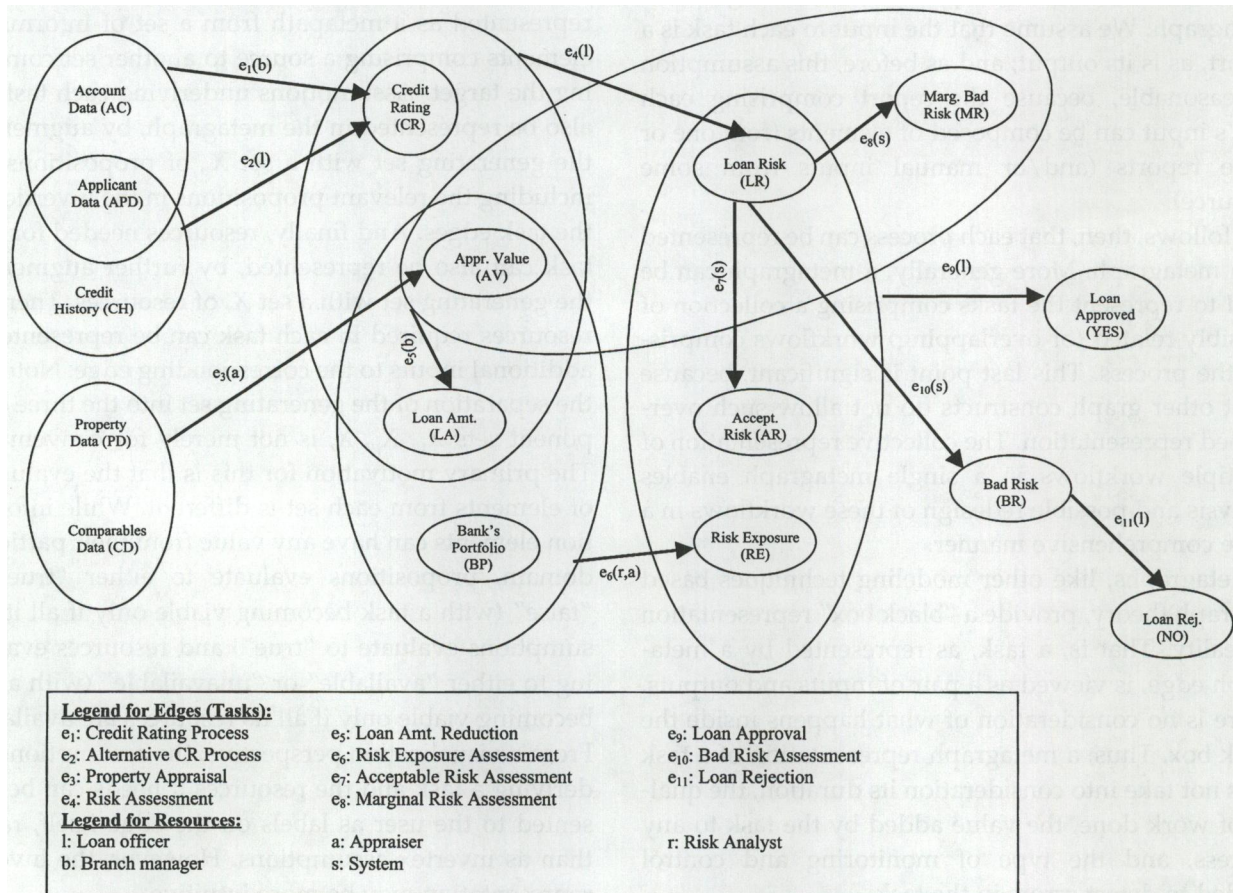
In this section we present three different types of analyses that can be performed using metagraph models of workflow systems. These analyses involve interactions among information elements (mediated by tasks), interactions among tasks (mediated by information elements), and interactions among resources (mediated by both information elements and tasks). To illustrate these ideas, we first present an example.

4.2. An Example

Consider the example illustrated in Figure 4, of a workflow process that determines whether an application for a property loan is to be accepted or rejected. The workflow process is modeled as a conditional metagraph. The information elements in the workflow process, represented by elements in the conditional metagraph, are as follows:

- AC: account data relevant to the applicant;
- APD: data about the applicant contained in the application;
- CH: credit history of the applicant;
- PD: data about the property for which the loan is being sought;
- CD: data about comparable properties;
- CR: applicant's credit rating;
- AV: the appraised value of the property;
- LA: the amount of the loan;
- BP: the current bank portfolio of loans;
- LR: the level of risk associated with the loan;
- RE: the bank's current risk exposure;
- YES: a statement that the application is approved;
- BR: a statement that the loan being applied for is a bad risk;
- NO: a statement that the application is rejected.

Figure 4 Metagraph of Loan Evaluation Process



The tasks in the workflow depend not only on the information elements identified above but also on the following assumptions:

- *AR*: whether the level of risk is acceptable;
- *MR*: whether the loan application represents a marginally bad risk.

There are eleven tasks in the workflow process, as follows:

- e_1 : The branch manager uses account data and applicant data to calculate the applicant's credit rating.
- e_2 : The loan officer uses applicant data and the applicant's credit history to calculate the applicant's credit rating.
- e_3 : The property appraiser uses data about the property along with data about comparable properties to calculate the appraised value of the property.
- e_4 : The loan officer uses the applicant's credit rating, the appraised value of the property, and the loan amount to calculate the level of risk associated with the loan.
- e_5 : If the risk of the loan is determined to be a bad risk, the branch manager uses the appraised value of the property and the level of risk associated with the loan to calculate a new loan amount.
- e_6 : The risk analyst and the property appraiser use the appraised value of the property, the loan amount, and the current bank portfolio of loans to calculate the bank's current risk exposure.
- e_7 : The system examines the risk associated with the loan and performs the calculations needed to determine whether the risk is acceptable.
- e_8 : The system examines the risk associated with the loan and performs the calculations needed to determine whether the risk is a marginally bad risk.
- e_9 : If the level of risk is acceptable, the loan officer uses the risk associated with the loan and the bank's risk exposure to decide whether to approve the loan.
- e_{10} : The system examines the risk associated with the loan and performs the calculations needed to determine whether the loan application represents a bad risk.
- e_{11} : If the loan application represents a bad risk, the loan officer rejects the application.

Finally, there are five resources used in the process, and these are shown in parentheses on the edge label

(although semantically they are treated as additional invertex elements), as follows:

- *a*: a property appraiser;
- *b*: the branch manager;
- *l*: a loan officer;
- *r*: a risk analyst;
- *s*: an automated system;

4.3. Analysis of Information Interactions

We now show how the properties of metagraphs can be applied to the analysis of interactions among information elements. The analysis is operationalized through the use of the A and A^* matrices.

First, the role of each information element x can be analyzed by examining the row and column corresponding to x in the matrices. For instance, each triple in the row corresponding to x in A^* represents a simple path from x to some element, and identifies the coinputs and cooutputs of the path. Similarly, each triple in the column corresponding to x represents a path from some element to x . A number of additional analyses can also be done from this information, such as the identification of necessary coinputs or tasks between any pair of elements x and y (coinputs and edges that appear in each triple in the corresponding cell in the A^* matrix), and the identification of any cycles through x (identified by $a_{ij}^* \neq \emptyset$). For our example in Figure 4, we may want to know if it is necessary to have the account data (*AC*) to determine the bank's risk exposure (*RE*). The visual representation may suggest that there is no connection between these elements. However, the cell in the A^* matrix corresponding to the row for account data (*AC*) and the column for risk exposure (*RE*) has the value $a_{AC,RE} = \{\langle \{AV\}, \{CR, LA, LR, MR\}, \langle e_1, e_4, e_8, e_5, e_6 \rangle \rangle\}$, indicating that if the appraisal value (*AV*) as well as the account data is known, then risk exposure can be computed using the tasks corresponding to the list $\langle e_1, e_4, e_8, e_5, e_6 \rangle$, which would also yield the applicant's credit rating (*CR*), the loan amount (*LA*), the loan risk (*LR*) and whether the case involves marginally bad risk (*MR*). (We do not show the entire matrix here; only relevant cells are described as needed.) This shows that indeed *AC* does affect *RE*. Examination of the cooutput also reveals that *MR* is one of the intermediate outputs. This indicates that the dependence of *RE* on *AC* occurs only when the

case is evaluated as marginally risky. Such analysis can facilitate scheduling tasks in workflows, as well as resource allocation.

Because a process is composed of a set of tasks that connect a source set of information elements to a target set of information elements, each workflow can be represented by a metapath from the underlying process's source to its target. This implies that a variety of metapath analysis mechanisms can be applied to workflow analysis. For instance, given a source and target, the A^* matrix can be used to identify the possible workflows for the process, using algorithms developed in (Basu and Blanning 1994b). In other words, a metapath search can help us to determine whether a process is functionally complete or not. That is, if there is no metapath available from the process source to the process target, then additional tasks will have to be included, or else some or all of the component tasks will have to be redesigned. For instance, we might assume that given all the information in a completed loan application, such as account data (*AC*), applicant data (*APD*), credit history (*CH*), and property data (*PD*), the loan process could be completed (i.e., the values of the loan amount (*LA*) and *YES* could be computed). However, if we try to construct a metapath from $\{AC, APD, CH, PD\}$ to $\{LA, YES\}$, we would fail. Based on visual inspection, we might add the data on comparable properties (*CD*), and try again. However, even then, no metapath is found. The element *BP*, representing information about the bank's existing loan portfolio, persists as a coinput. This analysis indicates the need for *BP* as an essential input for successful loan evaluation. On the other hand, there is a metapath from the application data $\{AC, APD, CH, PD, CD\}$ to $\{NO\}$, indicating that a complete workflow for unacceptable cases can be completed without information about *BP*. Again, this conclusion can be useful in designing the process (e.g., *BP* is acquired only after the loan data lead to instantiation of acceptable risk (*AR*)). Furthermore, if there are multiple workflows possible for a given source and target, the concept of a dominant metapath can be applied to facilitate choice among them (Basu et al. 1997). Conversely, given a specific workflow, the same property can be used to determine whether this workflow is efficient (i.e., whether it corresponds to a dominant metapath), or whether some

alternative workflow may be preferable. Again, in the loan process the loan amount adjustment task is superfluous, unless the application reflects marginally bad risk (i.e., *MR* is evaluated as True and *AR* is not). Additional analyses include identification of source-dominant metapaths for a given target set (i.e., metapaths that require a minimal source set for the given target set, even if each such metapath includes additional edges). The role of the applicant's credit history (*CH*) in the workflow (metapath $\{e_1, e_2, e_3, e_4, e_6, e_7, e_9\}$) to loan acceptance (*YES*) exemplifies this case. Initially, we might assume that the source for this workflow is $\{AC, APD, BP, CH, CD, LA, PD\}$ but then discover that this metapath is dominated by the metapath $\{e_1, e_3, e_4, e_6, e_7, e_9\}$, with source $\{AC, APD, BP, CD, LA, PD\}$, which does not include *CH*.

The projection operation on a metagraph is another useful construct for analyzing workflows. By focusing on a specific subset of information elements $X'_v \subset X_v$ it displays the relationships among these elements by identifying the processes that relate these elements. From a visualization standpoint, the projection view of a workflow metagraph is valuable, since it focuses attention on a few important elements and tasks. At the same time, the analytical structure underlying the view, in terms of the composition of each projected edge (Basu et al. 1997), enables identification of the structure of specific tasks making up the projection. For instance, a projection over appraised value (*AV*), bank portfolio (*BP*), loan risk (*LR*), and *YES* would show an edge from $\{AV, BP, LR\}$ to $\{YES\}$ (propositions such as *AR* and *MR* can be hidden in projections, to simplify visualization), which illustrates that under some conditions, the factors determining loan acceptance are the appraised value of the property, the loan risk, and the bank's current loan portfolio. This is difficult to ascertain visually from the detailed metagraph, yet is obvious in the projection.

Another abstraction that proves valuable in workflow analysis is a context. Recall that the assumptions applicable to each task in a workflow can be included as propositions in the metagraph representation. Given a set T of known true propositions and a set F of known false propositions, the applicable workflows under these conditions can be identified by constructing the corresponding context of the metagraph, as

shown in Basu and Blanning (1998). The context metagraph can then be analyzed in all the ways described above. As with the projection operation, the context metagraph provides a means for simplifying complex workflows and focusing attention to relevant tasks and elements.³ Contexts also help process designers avoid unpleasant situations. For example, given a process defined by a specific source and target, verifying functional completeness of the process in various relevant contexts (by ensuring the existence of relevant metapaths in each context) can help avoid nasty surprises such as the process failing under certain conditions. In our example, we can construct contexts for the different levels of loan risk represented by the propositions *AR*, *MR*, and *BR*, and examine the three alternative workflows that result for the loan evaluation process.

Thus far, we have answered questions 1, 2, and 3 posed in Table 1. In addition, other connectivity properties, such as cycles and bridges, can be analyzed using our algebraic approach. For example, the cycle through the edges $\{e_4, e_5\}$ can be identified using the diagonal elements of the A^* matrix, and the fact that the edge e_4 is a bridge from the application data to either loan outcome can be algorithmically determined. Thus, any software for workflow analysis based on metagraphs would have a user-friendly GUI for purposes of visualization, but its operation would use structured procedures based on algebraic representations of metagraphs which could be used to answer questions such as those we have discussed in this section.

4.4. Analysis of Task Interactions

We now turn to the analysis of tasks in workflows. Recall that tasks are represented in the metagraph as edges and appear as the third component of each triple in the adjacency matrix of the metagraph.

In workflow analysis, a number of questions about tasks and their role can arise, as exemplified by the questions listed in § 1. For such analysis it is useful, from both visualization and analytical viewpoints, to have a task-centric view of the workflow system as opposed to the element-centric view considered so far. Put another way, it would be useful to have tasks as

³In fact, the operations of creating projections and contexts are commutative (Basu and Blanning 1998).

elements of the generating set, and edges linking sets of related tasks. This can be achieved using a simplified version of the inverse metagraph described in § 3. In the context of workflow metagraphs, we call the resulting construct the *task interaction metagraph* (TIM) for the workflow system.

To construct the task interaction metagraph using Procedure Inverse in § 3, we modify the procedure as follows:

(a) Step 3 of the procedure is not used because we represent only interactions among tasks in the TIM. Thus, pure inputs and pure outputs are excluded.

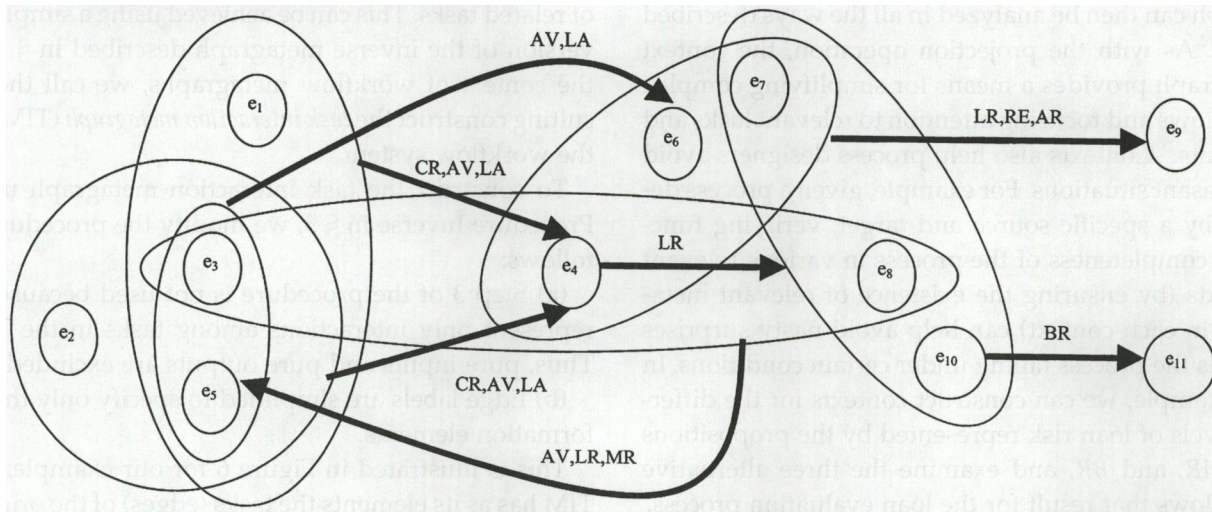
(b) Edge labels are simplified to specify only the information elements.

This is illustrated in Figure 6 for our example. The TIM has as its elements the tasks (edges) of the original metagraph, and each edge represents a situation in which one or more tasks (in its invertex) communicates with one or more tasks (in its outvertex) by providing information to them. In Figure 6, the edge from e_4 to e_7, e_8, e_{10} illustrates the dependence of the latter three tasks upon e_4 for the value of the loan risk (*LR*). From the closure of the TIM's adjacency matrix, we can identify the metapath $\{LR, BR\}$ from e_4 to e_{11} , which shows that once e_4 is executed, the edges in the metapath can be executed without interaction with any other tasks. On the other hand, the lack of a metapath from e_4 to e_9 indicates that other tasks besides e_4 have to provide information in order to execute e_9 . However, there is a

Figure 5 Incidence Matrix for the Loan Evaluation Metagraph in Figure 4

G	e ₁	e ₂	e ₃	e ₄	e ₅	e ₆	e ₇	e ₈	e ₉	e ₁₀	e ₁₁
AC	-1										
APD	-1	-1									
CH		-1									
PD			-1								
CD			-1								
BP						-1					
YES									+1		
NO											+1
CR	+1	+1		-1							
LA				-1	+1	-1					
AV			+1	-1	-1	-1					
LR				+1	-1		-1	-1	-1	-1	
AR							+1		-1		
RE						+1					
MR					-1			+1			
BR										+1	-1
b	-1				-1						
l		-1		-1					-1		-1
a			-1			-1					
r						-1					
s							-1	-1		-1	

Figure 6 Task Interaction Metagraph for Process in Figure 4



metapath from $\{e_1, e_3, e_5\}$ to $\{e_9\}$. This shows that the edges in that metapath can be used to enable e_9 , without interaction with any other tasks. Similarly, the cycle through the edges e_4 and e_5 reveals that these tasks may be executed multiple times in workflows (metapaths) in which they both appear. Such analysis, as exemplified by questions 1 and 2 in § 1, can help in designing workflows because coordination of tasks is necessary only when there are dependencies among them.

The TIM can also be useful in analyzing the impact of one or more tasks failing during a process. Another related question is whether a particular task is essential for a workflow. Because a task could produce multiple outputs, each of which is used in possibly several other tasks, such questions cannot always be answered simply through the visual representation of either the process metagraph or its TIM. However, by testing whether a given workflow metapath is dominant or not from the closure matrices for either representation, these questions can be answered in a structured manner. Further, if we want to know whether a particular task e_{pi} in a workflow $\{e_{p1}, e_{p2}, \dots, e_{pm}\}$ would be disabled by the failure of another task e_j , the closure of the TIM's adjacency matrix can be utilized. If all the paths in the column corresponding to e_j and rows $\{e_{p1}, e_{p2}, \dots, e_{pm}\}$ contain e_j , then indeed e_j is essential for e_{pi} . For example, in the TIM in Figure 6, every path from $\{e_1, e_3, e_5\}$

to e_9 contains e_4 , which is thus essential for the workflow from $\{e_1, e_3, e_5\}$ to $\{e_9\}$.

We have now answered questions 4 and 5 posed in Table 1. The strength of metagraph-based task analysis in workflows is that both visual inspection and algebraic analysis of workflow tasks can be achieved using the same operations and methods as those used for analysis of information elements and their interactions. In other words, the approach integrates functional and informational analysis of a system (Curtis et al. 1992), which is difficult using traditional tools for process analysis.

4.5. Analysis of Resource Interactions

We now turn to the analysis of the role of each resource in the workflow, and interactions between different resources. A resource may be human (a person, group, team, or task force), or equipment (e.g., computers, software packages or programs, and databases). Furthermore, a resource may be a particular entity (e.g., a specific person) or a class (e.g., a functional role, such as a financial analyst). There is a many-to-many mapping between tasks and resources—each task may require several resources, and each resource may be required in several tasks.

An important part of business process design is the effective allocation of resources to tasks. In addressing this allocation problem, there are two dimensions. The

first is the functional interaction among different resources, the tasks they perform, and the information elements they use and produce. These considerations and their analysis impact the design of processes and their workflows. The second dimension is the temporal constraints applicable to the interactions that impact the operational control of workflows during their execution. This is important for monitoring and control of workflow execution (at run time), in addition to workflow design. The approach presented in this paper addresses only the first dimension because we do not include temporal attributes for tasks (although our approach can be extended toward this end with attributed metagraphs). Thus, our focus is on understanding how different resources interact with each other, the tasks through which these interactions occur, and the information they exchange.

Interactions among resources can be specified by a *resource interaction metagraph* (RIM), which shows where resources provide information to each other through a sequence of two successive tasks. This is accomplished by using the element flow metagraph defined in § 3. If the element set X' for the element flow metagraph is restricted to the set of resources, then the result is in fact the RIM. For our example, the G_1 and G_2 matrices for Procedure EFM correspond to the sets of rows in Figure 5 denoted as G_1 and G_2 . The R matrix that results from the first step is as follows, and the result of the procedure is the RIM illustrated in Figure 7.

Using this metagraph we can determine which resources depend on other resources to provide information. That is, we can identify cases in which a resource is a member of a task that provides information to a task of which the second resource is a member. For example, for the loan evaluation process, the branch manager interacts with the loan officer by providing the latter with the applicant's credit rating for the risk assessment task; the loan officer in turn performs the risk assessment and returns the risk value of the loan to the branch manager. In addition, when the branch manager performs the loan amount reduction task, she also provides a loan amount to the loan officer for the loan risk assessment. This interaction can be

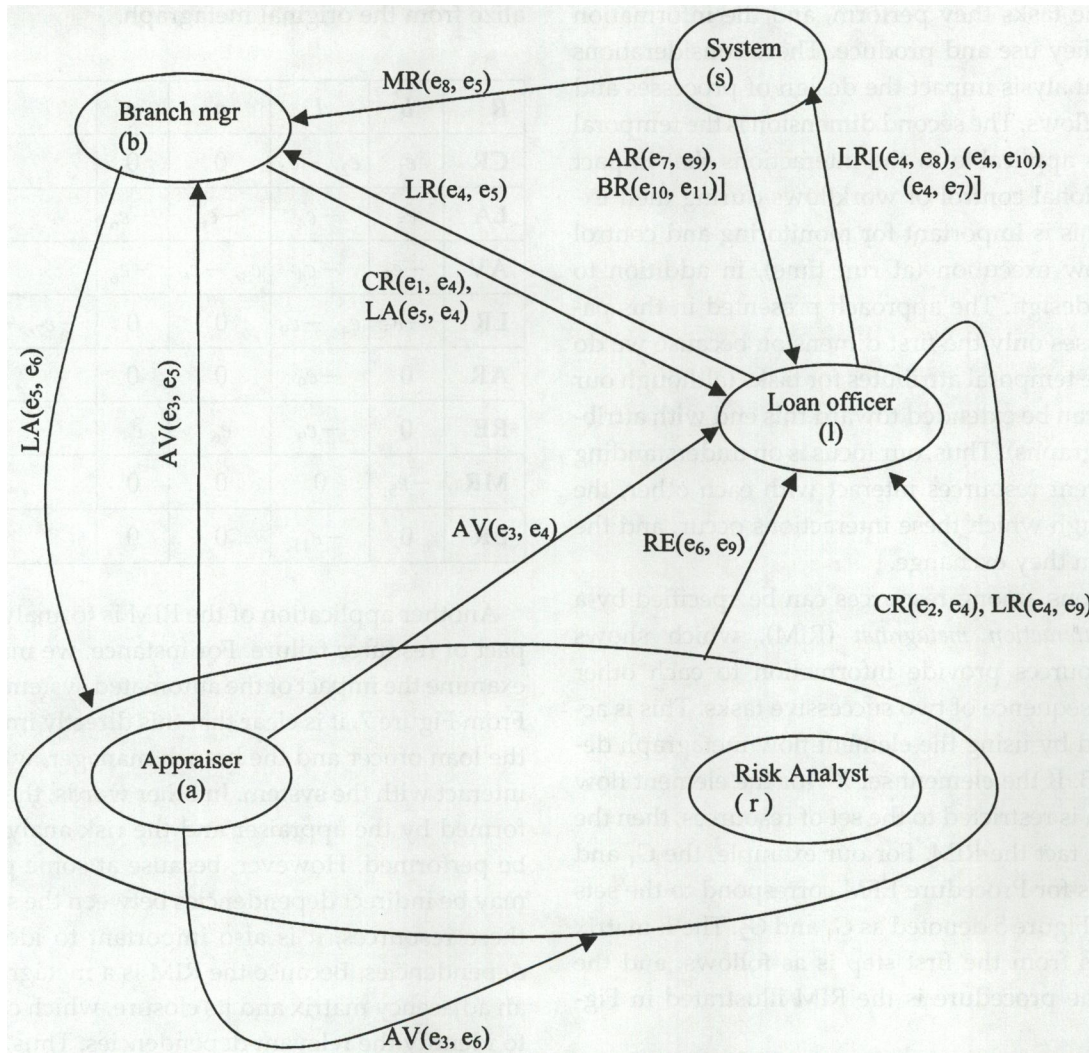
easily visualized from the RIM but is difficult to visualize from the original metagraph.

R	b	l	a	r	s
CR	e_1	$e_2, -e_4$	0	0	0
LA	e_5	$-e_4$	$-e_6$	$-e_6$	0
AV	$-e_5$	$-e_4$	$e_3, -e_6$	$-e_6$	0
LR	$-e_5$	$e_4, -e_9$	0	0	$-e_7, -e_8, -e_{10}$
AR	0	$-e_9$	0	0	e_7
RE	0	$-e_9$	e_6	e_6	0
MR	$-e_5$	0	0	0	e_8
BR	0	$-e_{11}$	0	0	e_{10}

Another application of the RIM is to analyze the impact of resource failure. For instance, we may want to examine the impact of the automated system (s) failing. From Figure 7, it is clear that this directly impacts only the loan officer and the branch manager, who directly interact with the system. In other words, the tasks performed by the appraiser and the risk analyst can still be performed. However, because at some point there may be indirect dependencies between the system and these resources, it is also important to identify such dependencies. Because the RIM is a metagraph, it has an adjacency matrix and its closure, which can be used to identify the relevant dependencies. Thus, if the row in the closure matrix corresponding to the system has any entries in the columns for the appraiser and the risk analyst, then indeed there is a dependency, and at some point these resources will be impacted by the system's failure. This is indeed the case, as indicated for instance by the path from s to $\{a, r\}$ through b (the information flows $MR(e_8, e_5)$, $LA(e_5, e_6)$). There are other paths too, such as the path through l and b (the information flows $AR(e_7, e_9)$, $BR(e_{10}, e_{11})$), and another involving the loop through l . Once these dependencies are identified, their roles can be analyzed using either the original metagraph, or the RIM.

In this subsection, we have presented a third view of workflow metagraphs, the *resource interaction meta-*

Figure 7 Resource Interaction Metagraph for Loan Process in Figure 4



graph. This view, the resource task assignment matrix, and the transformation operation used to generate it from the original workflow metagraph are new metagraph constructs. Using this view, we can answer a variety of important questions about the role of resources in the workflow and its underlying process, including questions 6 and 7 posed in Table 1. As with the analysis of tasks discussed in the previous subsection, while the original workflow metagraph can be used to address such questions, the RIM significantly enhances both visualization as well as analysis of resource interactions, which in turn can lead to better workflows and process design.

4.6. Interactions Among Different Types of Components

The previous subsections focused on questions about the interactions among components of the same type, that is among information elements, tasks, and resources, respectively. We now examine an important additional dimension, namely questions that span different component types.

For instance, we may want to determine the effect of a particular resource's unavailability upon the feasibility of one or more workflows. In the loan process, if the risk analyst (*r*) were unavailable, could the loan evaluation still be completed? We answer this question

by restricting the metagraph in Figure 4 to the context where r is unavailable. This results in edge e_6 being disabled. If we now try to construct metapaths to {YES} and {NO}, respectively, from {AC, APD, PD, CH, BP}, we find that we can still complete the workflow for rejected loans but not for approved loans. Thus, the risk analyst is essential for making positive loan decisions but not for negative ones.

On the other hand, if the automated system resource (s) were unavailable, then the edges e_7 , e_8 , and e_9 would be disabled, which would disconnect the source and target for the process, rendering the process infeasible. This is important, because it indicates to the process designer that either the resource s should be designed to be highly reliable, or the process should be enhanced with additional tasks and/or resources that could be used to reduce the criticality of the automated system.

Another question that may arise is: what resources are responsible for a given set of information elements? For example, assume that bank management raised a question about the loan risk assessment values. To determine all the resources that contributed to the valuation of this element, we could examine the process metagraph visually and conclude that the branch manager (b), loan officer (l), and appraiser (a) were the relevant resources. However, by examining the cells a_{iLR}^* in the A^* matrix for all resource rows i and the column LR , we would find that the automated system s is also relevant. Especially in complex processes with many tasks, such analysis can help determine accountability for tasks and outcomes in a systematic manner.

We have now answered questions 8 and 9 posed in Table 1.

5. Discussion and Conclusion

We have shown that metagraphs allow us to examine the relationships within and among three different but related aspects of workflow systems: information elements, tasks, and resources. This is accomplished by constructing views of metagraphs that are themselves metagraphs. Therefore, the same set of concepts and operations involving simple paths, coinputs, cooutputs, metapaths, projections, and contexts can be used regardless of which aspects of workflow systems are to be analyzed. In addition, the correspondence between the visualization and algebraic capabilities of

metagraphs not only allows algebraic operators to supplement or even to substitute for visualization, but also to form a foundation for the construction of computer-based tools to support workflow management.

Having described this metagraph-based approach to workflow analysis, it is useful to revisit a comparison with methods based on Petri nets and statecharts. Both these latter approaches support formal analysis of processes. As stated earlier, they are both included in the UML. Collectively, the various constructs included in the UML provide a powerful set of resources for process modeling. However, our approach has the advantage that it uses a single representational construct, which is also usable for traditional system modeling, of system components such as data, models and rules (Basu and Blanning 1994a). Furthermore, as discussed in Curtis et al. (1992), effective process modeling requires support for multiple perspectives, and metagraphs provide a powerful and efficient solution for this purpose.

This versatility yields significant benefits in two ways. First, metagraph-based software tools for process analysis can be more compact and efficient than equivalent UML tools, which would require implementation of a variety of operators and algorithms for the various constructs. Second, analysts and designers using metagraph-based tools do not need to learn several different visualization constructs because they navigate between alternative views of a process (the element-centric, task-centric, and resource-centric views, respectively). In other words, we would expect users of metagraph-based tools to encounter shorter learning curves. This would be true even if additional tools are needed for specialized analyses because the composite system would still use fewer constructs than other approaches such as UML require.

Metagraphs may also form a foundation for work in related areas of workflow management. One such area is the scheduling of tasks, which would require the labeling of tasks (metagraph edges) with quantitative attributes such as task duration, and the development of scheduling algorithms. Another area is workflow monitoring and control. In this case, it is important to know how errors and the consequent need for corrections and rework propagate through a process, and to

identify critical points where monitoring may be essential to prevent major problems. While the existing formulation of metagraphs may not be adequate to solve these problems completely, metagraph analysis may still be useful for problem detection and evaluation.

A metagraph-based framework for workflow analysis, based on the results presented here and on some of these related areas, may also form the basis for a software system for workflow analysis and process design. As discussed in § 2, existing workflow software focuses primarily on visualization of relationships among components of workflow systems and does not support analysis of the type described in this paper (Khoshafian and Buckiewicz 1995, Koulopoulos 1995). A metagraph-based tool would contain modules for constructing models of workflow systems based on information elements, edges, assumptions, and resources and for constructing metapaths, projections, contexts, and the transformations described in this paper (Basu and Blanning 1996). Such a system might be useful not only for analysis of workflows during process design, but also for implementation, control, and modification of processes.

Appendix A. Metagraph Definitions

DEFINITION I (from Basu and Blanning 1994a). Given a finite generating set $X = \{x_i, i = 1, \dots, J\}$, a metagraph is an ordered pair $S = \langle X, E \rangle$ in which E is set of edges $\{e_k, k = 1, \dots, K\}$. Each edge is an ordered pair $e_k = \langle V_k, W_k \rangle$, in which $V_k \subseteq X$ is the *invertex* of the edge e_k and $W_k \subseteq X$ is the *outvertex*. The *coinput* of any $x \in V_k$ is $V_k \setminus \{x\}$ and the *cooutput* of any $x \in W_k$ is $W_k \setminus \{x\}$. Also $V_k \cup W_k \neq \emptyset, \forall k$.

DEFINITION II. Given a metagraph $S = \langle X, E \rangle$, a *simple path* from an element $x \in X$ to an element $x' \in X$ is a sequence of edges $h(x, x') = \langle e'_l, l = 1, \dots, L \rangle$, such that $x \in V_1, x' \in W_L$, and $W_i \cap V_{i+1} \neq \emptyset, \forall i = 1, \dots, L - 1$. The *coinput* of x is

$$\left(\bigcup_{l=1}^L V_l \setminus \bigcup_{l=1}^L W_l \right) \setminus \{x\},$$

and the *cooutput* of x' is

$$\bigcup_{l=1}^L W_l \setminus \{x'\}.$$

The *length* of a simple path is the number of edges in the path; thus, the length of $h(x, x')$ is L . An edge is a simple path of length one. A *cycle* is a simple path from an element to itself; thus it is of the form $h(x, x)$ for some $x \in X$.

DEFINITION III. Given a metagraph $S = \langle X, E \rangle$, a *metapath* M

(B, C) from a source $B \subseteq X$ to a target $C \subseteq X$ is a set of edges $\{e'_l, l = 1, \dots, L\}$ such that⁴

1. There is a set of simple paths $\{h_m(x'_m, x''_m), m = 1, \dots, M\}$ with $x'_m \in B, x''_m \in C \forall m$, such that $M(B, C) = \bigcup_{m=1}^M \text{Set}(h^m(x'_m, x''_m))$.
2. $\left(\bigcup_{l=1}^L V_l \setminus \bigcup_{l=1}^L W_l \right) \subseteq B$.
3. $C \subseteq \bigcup_{l=1}^L W_l$.

A metapath $M(B, C)$ is said to be *dominant* if there is no metapath to C from any proper subset of B and no proper subset $M' \subset M$, which is also a metapath from B to C (Basu and Blanning 1994b).

Using the adjacency matrix A of a metagraph and its powers A^n , we can identify simple paths of various lengths from any element x_i to any other element x_j (Basu and Blanning 1994a). Each member of A^n, a_{ij}^n , is a set of zero or more triples, one for each simple path of length n connecting x_i to x_j . The first component of the triple is the coinput of x_i in the path, the second component is the cooutput of x_j , and the third component is the path. For the metagraph in Figure 1, $a_{13} = \{\langle \{x_2\}, \{x_4\}, \langle e_1 \rangle \rangle\}$, while $a_{76}^2 = \{\langle \{x_2, x_5\}, \{x_3, x_4, x_8\}, \langle e_1, e_3 \rangle \rangle, \langle \{x_3, x_4\}, \{x_5, x_8\}, \langle e_2, e_3 \rangle \rangle\}$, corresponding to the two simple paths from x_1 to x_6 .

The closure A^* of the adjacency matrix is $A + A^2 + A^3 + \dots$. This matrix discloses all paths of any length connecting any two elements, and it can also be used to identify metapaths (Basu and Blanning 1997b). Because a metapath is the union of the edges in a set of simple paths, metapaths can be formed by combining triples in a_{ij}^n for all $x_i \in B$ and $x_j \in C$ in such a way that the conditions of the definition of a metapath are met (Basu and Blanning 1997b).

We now describe some abstractions of a metagraph. For this, we first define a conditional metagraph, in which the generating set is partitioned into two subsets. The first is a set of variables, denoted X_v , and the second is a set of propositional statements (such as "the inflation rate is less than five percent," which can be either *true* or *false*), denoted X_p .

DEFINITION IV (Basu and Blanning 1998). A *conditional metagraph* is a metagraph $S = \langle X, E \rangle$, in which:

- 1) $\forall e' \in E, V' \cup W' \neq \emptyset$ and $V' \cap W' = \emptyset$,
- 2) $X = X_v \cup X_p$ with $X_v \cap X_p = \emptyset$ such that $\forall p \in X_p$,
 - a) $\forall e' \in E$, if $p \in W'$, then $W' = \{p\}$,
 - b) $\forall e' \in E, \forall e'' \in E$, if $p \in V'$ and $W'' = \{p\}$, then $V' \cap W' = \emptyset$.

DEFINITION V. Given a conditional metagraph $S = \langle X_v \cup X_p, E \rangle$, a source $B \subseteq X_v$, and a target $C \subseteq X_v$, a *conditional metapath* is a set of edges $M(B, C) = \{e'_l, l = 1, \dots, L\}$ forming a metapath from $B \cup X_p$ to C .

DEFINITION VI (Basu and Blanning 1998). Given a conditional metagraph $S = \langle X_v \cup X_p, E \rangle$, a set of assumptions $P \subseteq X_p$ that are known

⁴A set is denoted by $\{a, b, c, \dots\}$ and a sequence by $\langle a, b, c, \dots \rangle$. Also, the function $\text{Set}(\)$ converts a sequence into a set; that is, $\text{Set}(\langle a, b, c, \dots \rangle) = \{a, b, c, \dots\}$.

to be true, and a set of assumptions $Q \subseteq X_p$ that are known to be false, a context $K(P, Q, S)$ is a conditional metagraph derived from S as follows:

1. For any edge $e' \in E$ containing a proposition $p \in P$, simplify the edge by deleting p ; if the resulting edge has a null in- or out-vertex, delete the edge.

2. For any edge $e' \in E$ containing an proposition $q \in Q$ in either vertex, delete the edge (only the edge and q are deleted, not the other elements in the edge's vertices).

Another type of higher level view is a projection. In projecting a metagraph, a user specifies a subset of the generating set, and the projection is a simplified metagraph that contains only the relevant elements and edges.

DEFINITION VII (Basu and Blanning 1998). Given a conditional metagraph $S = \langle X_v \cup X_p, E \rangle$, a projection of S over the set $X' \subseteq X_v$ is a conditional metagraph $N(X', S) = \langle X' \cup X_p, E' \rangle$ such that $e' \in E'$ if there is a dominant metapath from $V_{e'}$ to $W_{e'}$ in S .

Finally, we note that these two types of views, context and projection, are commutative—that is, $N(X', K(P, Q, S)) = K(P, Q, N(X', S))$ (Basu and Blanning 1998).

Appendix B. The Operator $\otimes$

Let A be an $m \times n$ matrix with row indices $x_1, \dots, x_m$ and column indices $y_1, \dots, y_n$. Let B be an $n \times p$ matrix with row elements $y_1, \dots, y_n$ and column indices $z_1, \dots, z_p$. Then the operation $A \otimes B$ results in an $m \times p$ matrix C with row indices $x_1, \dots, x_m$ and column indices $z_1, \dots, z_p$, such that $c_{ij} = \bigcup_{k=1, \dots, n} a_{ik} \oplus b_{kj}$

and

$$a_{ik} \oplus b_{kj} = \begin{cases} y_k & \text{if } a_{ik} = 1 \text{ and } b_{kj} = -1, \\ -y_k & \text{if } a_{ik} = -1 \text{ and } b_{kj} = -1, \\ 0 & \text{otherwise.} \end{cases}$$

References

- van der Aalst, W. M. P. 1995. A class of Petri nets for modeling and analyzing business processes. Computing Science Reports 95/26, Eindhoven University of Technology.
- Barua, A., C. H. Sophie Lee, A. B. Whinston. 1996. The calculus of reengineering. *Inform. Systems Res.* 7(4) 409–428.
- Basu, A., R. W. Blanning. 1992. Metagraphs and Petri nets in model management. *Proc. Workshop Inform. Tech. Systems* 64–73.
- . 1994a. Metagraphs: A tool for modeling decision support systems. *Management Sci.* 40(12) 1579–1600.
- . 1994b. Model integration using metagraphs. *Inform. Systems Res.* 5(3) 195–218.
- . 1996. A metagraph-based DSS analysis workbench. *Proc. Twenty-Ninth Hawaii Internat. Conf. System Sci.* Vol. II, (January) Maui, HI. 386–395.
- . 1997. Metagraph transformations and workflow analysis. *Proc. Thirtieth Hawaii Internat. Conf. System Sci.* Vol. IV, (January) Maui, HI. 359–366.
- . 1998. The analysis of assumptions in model bases using metagraphs. *Management Sci.* 44(7) 982–995.

- , A. Shtub. 1997. Metagraphs in hierarchical modeling. *Management Sci.* 43(5) 623–639.
- Berge, C. 1985. *Graphs*, 2nd ed. North-Holland, Amsterdam. The Netherlands.
- . 1989. *Hypergraphs: Combinations of Finite Sets*. North-Holland, Amsterdam. The Netherlands.
- Booch, G., J. Rumbaugh, I. Jacobson. 1998. *The Unified Modeling Language Reference Manual*. Addison-Wesley, New York.
- Curtis, B., M. I. Kellner, J. Over. 1992. Process modeling. *Comm ACM* 35(9) 75–90.
- Datta, A., A. Basu. 1997. Workflow analysis: A transaction oriented approach. Working paper, Owen Graduate School of Management, Vanderbilt University, TN.
- Datamation. 1995. Datamation's feature summary: High-end workflow. *Datamation* 41(15).
- Davenport, T. H. 1993. *Process Innovation: Reengineering Work Through Information Technology*. Harvard Business School Press, Boston, MA.
- Georgakopoulos, D., M. Hornick, A. Sheth. 1995. An overview of workflow management: From process modeling to workflow automation infrastructure. *Distributed and Parallel Databases* 3 119–153.
- Hammer, M. 1996. *Beyond Reengineering*. Harper Business, New York.
- , J. Champy. 1993. *Reengineering the Corporation: A Manifesto for Business Revolution*. Harper Business, New York.
- Harel, D. 1987. Statecharts: A visual formalism for complex systems. *Sci Comput. Programming* 8 pp. 231–274.
- , A. Naamad, A. Pneuli, M. Politi, R. Sherman, A. Shtull-Tauring. 1988. STATEMATE: A working environment for the development of complex, reactive systems. *Proc. Tenth IEEE Internat. Conf. Software Engrg.* Singapore.
- Jensen, K. 1992. Colored petri nets: Basic concepts, analysis methods and practical uses. *EATCS Monographs on Theoretical Comput. Sci.* Springer-Verlag, Berlin, Germany.
- Khoshafian, S., M. Buckiewicz. 1995. *Introduction to Groupware, Workflow, and Workgroup Computing*. Wiley, New York, 207–258.
- Kouloupoulos, T. M. 1995. *The Workflow Imperative*. van Nostrand Reinhold, New York.
- Kwan, M. Millie, P. R. Balasubramanian. 1997. Dynamic workflow management: A framework for modeling workflows. *Proc. Thirtieth Hawaii Internat. Conf. System Sci.* IV (January) Maui, HI. 367–376.
- Marschak, R. T. 1995. Workflow: Applying automation to group processes. D. Coleman, R. Khanna, eds., *Groupware: Technologies and Applications*. Prentice-Hall, Upper Saddle River, NJ, 71–77.
- Mohan, C., D. Agrawal, G. Alonso, A. El Abbadi, R. Guenther, M. Kamath. 1995. Exotica: A project on advanced transaction management and workflow systems. *SIGIS Bull. Special Issue: Bus. Process Reengrg.* 16(1) 45–50.
- Morschheuser, S., H. Raufer, C. Wargitsch. 1996. Challenges and solutions of document and workflow management in a manufacturing enterprise: A case study. *Proc. Twenty-Ninth Ann. Hawaii Internat. Conf. System Sci.* V (January) HI. 4–12.

- Murata, T. 1989. Petri nets: Properties, analysis and applications. *Proc. IEEE* 77(4) 541–580.
- Poyssick, G., S. Hannaford. 1996. *Workflow Reengineering*. Adobe Press, Mountain View, CA.
- Silver, B. 1995. Automating the business environment. L. Fischer, ed. *The Workflow Paradigm*, 2nd ed. Future Strategies, Lighthouse Point, 173–195, (also Appendix C: “Vendor Directory,” 315–342).
- Swenson, K. D. 1995. Workflow management standards and interoperability. L. Fischer ed., *The Workflow Paradigm* (2nd ed.). Future Strategies, Lighthouse Point, 25–36.
- Thomson, V. J. 1995. Process monitoring for continuous improvement. J. Browne, D. O’Sullivan, eds., *Reengineering the Enterprise*. Chapman and Hall, London, UK.
- WFMC. 1994. Workflow reference model. Technical Report, Workflow Management Coalition, Brussels, Belgium.

Steven O. Kimbrough, Associate Editor. This paper was received on February 14, 1996, and was with the authors 9 months for 3 revisions.